Claude Managed Agents

Guía práctica del cookbook:
de un hola mundo a dynamic workflows

por Daniel Ávila

Octubre 2026

github.com/davila7/claude-managed-agents-cookbook

1. ¿Qué es Claude Managed Agents?

Claude Managed Agents (CMA) es el arnés de agentes de Anthropic, ya construido y ejecutado en su infraestructura. En vez de programar tu propio loop (llamar al modelo, ejecutar herramientas, volver a llamar al modelo), le das a Claude un **sandbox en la nube** donde lee archivos, ejecuta comandos, navega la web y corre código. Tu aplicación solo **envía eventos** y **recibe un stream** con lo que el agente va haciendo. Anthropic se ocupa del resto: herramientas, caché de prompts, compactación de contexto y reintentos.

	Messages API	Claude Managed Agents
Qué es	Acceso directo al modelo	Arnés de agente + infraestructura gestionada
Ideal para	Loops propios, control fino	Tareas largas (minutos u horas) y trabajo asíncrono
Tú construyes	El loop, las tools, el runtime	Solo tu app: envías eventos y lees el stream

Cuatro conceptos lo explican todo:

Concepto	Qué es	Ciclo de vida
Environment	La plantilla del sandbox: contenedor cloud y red	Se crea una vez y se reutiliza
Agent	Configuración versionada: modelo, system prompt, tools	Se crea una vez; cada cambio crea una versión
Session	Un agente corriendo en un environment para una tarea	Una por tarea o conversación
Events	Mensajes entre tu app y el agente, vía SSE	Muchos por sesión

CMA está en **beta**: todas las llamadas llevan el header `managed-agents-2026-04-01`, que el SDK y el CLI agregan solos. Viene habilitado por defecto en todas las cuentas de la API.

2. Un caso de uso: ¿cuándo usarlo?

Imagina el equipo legal de una empresa (en el repo se llama *Brightloom Inc.*). Cada trimestre revisa decenas de contratos de proveedores: fechas de renovación automática, plazos de aviso, topes de responsabilidad, cláusulas de cambio de control. Hoy alguien lo hace a mano, con una planilla.

Con CMA, la aplicación sube los contratos, crea una sesión y escribe una sola instrucción: *"Revisa los contratos y escribe un resumen y un informe de riesgos"*. El agente lee cada archivo, calcula fechas con bash, escribe `summary.csv` y `report.md`, y la aplicación los descarga. Si son cientos de contratos, el agente escribe un **workflow** que reparte el trabajo: un agente por contrato y otro que verifica cada resultado. Este es exactamente el ejemplo que el cookbook construye, paso a paso, entre los módulos 04 y 07.

Usa CMA cuando…	Mejor la Messages API cuando…
La tarea dura minutos u horas y encadena muchas herramientas	Necesitas una respuesta rápida en un chat
El agente necesita ejecutar código o trabajar con archivos en un sandbox	Quieres controlar cada paso de tu propio loop
El trabajo es asíncrono: lanzas la tarea y vuelves por el resultado	Tus herramientas viven solo en tu backend
Quieres estado persistente (historial y archivos) o varios agentes en paralelo	Necesitas Zero Data Retention o un BAA de HIPAA (CMA no es elegible)

3. Requisitos

- **Python 3.10 o superior.** Los notebooks usan `match`, que no existe en 3.9. Compruébalo con `python3 --version`.
- **Visual Studio Code** con las extensiones **Python** y **Jupyter** de Microsoft.
- **Git**, para clonar el repo.
- **Una API key** de `platform.claude.com/settings/keys`, con saldo en la cuenta.
- **El CLI `ant`, versión 1.40 o superior**, solo para los notebooks `-cli`: `brew install anthropics/tap/ant` en macOS; en Linux, descarga un release de GitHub.

Costo: los notebooks llaman a la API real. Los módulos 01–05 cuestan céntimos cada uno, el 06 unos US$0,35 y el 07 alrededor de US$1. Las sesiones del 06 y el 07 llevan un presupuesto como tope duro.

4. Setup en VS Code

1. Clona el repo y ábrelo.

```
git clone https://github.com/davila7/claude-managed-agents-cookbook.git
cd claude-managed-agents-cookbook
code .
```

2. Crea un entorno virtual. Desde la terminal de VS Code (*Terminal → New Terminal*), o con la paleta (`Cmd/Ctrl+Shift+P` → *Python: Create Environment → Venv*):

```
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -U "anthropic>=1.13" python-dotenv ipykernel
```

3. Configura tu API key. Hay un único `.env` en la **raíz** del repo, compartido por todos los notebooks. No crees uno por carpeta.

```
cp .env.example .env
# edita .env:  ANTHROPIC_API_KEY=sk-ant-...
```

4. Abre un notebook y elige el kernel. Abre, por ejemplo, `01-basics/hello_world.ipynb`. Arriba a la derecha, pulsa **Select Kernel → Python Environments → .venv**. Si el kernel elegido no es el del venv, verás `ModuleNotFoundError`.

5. Ejecuta celda por celda. `Shift+Enter` ejecuta una celda y pasa a la siguiente. Lee el texto entre celdas: explica qué hace cada paso. Evita *Run All* la primera vez; la gracia es ir mirando la Console entre una celda y la siguiente.

Problemas frecuentes

Síntoma	Solución
`ModuleNotFoundError: anthropic`	El kernel no es `.venv`: cámbialo en Select Kernel.
`ANTHROPIC_API_KEY not found`	Falta el `.env` en la raíz, o la key está vacía.
`APIConnectionError: Connection error.` en todas las llamadas	Tienes una versión vieja de `brotli`: `pip install -U "brotli>=1.2"` y reinicia el kernel.
`ant: command not found` en un notebook CLI	Instálalo y **reinicia el kernel**, o VS Code, para que tome el nuevo PATH.

5. Cómo está organizado el repo

Cada carpeta numerada es un módulo autocontenido, con un `README.md` y un notebook. Cada tema viene en **dos sabores**: primero el SDK de Python y luego una versión **-cli** que enseña lo mismo con el CLI `ant`. Los módulos se construyen uno sobre otro, así que conviene seguirlos en orden:

```
01 basics → 02 multi-turn → 03 tools & permissions → 04 files
→ 05 session control → 06 subagents → 07 dynamic workflows
```

Elemento	Para qué sirve
`NN-tema/`	Notebook con el SDK de Python. Es la forma de integrar CMA en tu aplicación.
`NN-tema-cli/`	El mismo flujo con `ant`: agentes y environments declarados en archivos (`agent.md`, `environment.yaml`) que se aplican con `ant apply`.
`sample-data/contracts/`	Seis contratos ficticios que usan los módulos 04, 06 y 07.
`claude-lock.json`	Lo crea `ant apply`: guarda los IDs de lo que creó, para no duplicarlo. Contiene IDs de tu organización, por eso el repo lo ignora.
`outputs/`	Archivos que los notebooks descargan de las sesiones. También ignorados por git.

SDK o CLI, ¿cuál hago? Si vas a escribir una aplicación, empieza por el SDK. El CLI brilla para el *plano de control*: definir agentes como archivos versionados en git, revisarlos en un pull request y aplicarlos desde CI. Puedes hacer solo los SDK y volver a los CLI después.

6. Sigue todo en platform.claude.com

La gracia de este cookbook es **ver en vivo** cómo se crea y ejecuta cada recurso. Ten la Console abierta al lado de VS Code, y después de cada celda que cree algo, refresca y búscalo por su ID (`env_…`, `agent_…`, `sesn_…`).

En la barra lateral, dentro de **Managed Agents**, la sección **Sessions** lista cada sesión con su estado, agente, tokens, costo y fecha. Al abrir una ves:

- **Timeline:** la actividad en el tiempo, con un carril por thread cuando hay varios agentes.
- **Transcript:** la conversación agrupada por llamada al modelo, con cada tool, su input y su resultado.
- **Inspector**, con cinco pestañas: **Session** (detalles, costo y presupuesto), **Events** (cada evento en JSON), **Tools** (llamadas y fallos), **Resources** (archivos montados y outputs) y **Threads** (cada agente, con su costo).

Desde la terminal, `ant beta:sessions connect <session-id>` sigue una sesión en vivo.

7. Paso a paso por módulo

No hace falta repetir el código: está en cada notebook, comentado. Esta es la guía de qué buscar en cada uno.

Módulo 01 · Basics

Qué aprendes	El flujo completo: environment → agent → session → stream de eventos. La regla más importante, **stream-first**: abre el stream antes de enviar el mensaje, o perderás los primeros eventos.
Qué ejecutas	Cinco celdas: crear environment, crear agente, crear sesión, y enviar un mensaje leyendo el stream.
Qué mirar en la Console	Tras cada celda, el recurso nuevo con su ID. El agente aparece en **versión 1**. En la sesión, el **Transcript** muestra la respuesta, y la pestaña **Events** todo lo que no imprimimos (`span.*`, `session.usage`…).
Tiempo y costo	1–2 minutos, céntimos.

Módulo 02 · Multi-turn

Qué aprendes	Una sesión guarda la conversación en el servidor: envías solo el mensaje nuevo, nunca el historial. Un helper `chat()` envuelve el patrón stream-first.
Qué ejecutas	Tres turnos sobre la misma sesión (Júpiter y sus lunas). La celda REPL es opcional e interactiva.
Qué mirar en la Console	Una **sola sesión** que acumula los tres turnos en el Transcript.
Tiempo y costo	2 minutos, céntimos.

Módulo 03 · Tools & Permissions

Qué aprendes	Tu código participa en el turno. Un **custom tool** (`lookup_stock`) que corre en tu sistema, y `bash` con `always_ask`: tú apruebas o rechazas cada comando. Leer `stop_reason`: un idle con `requires_action` no es el final.
Qué ejecutas	El agente consulta tres SKUs, escribe un CSV (aprobado) e intenta borrarlo (rechazado con un mensaje). Después se actualiza el agente a la política `auto`.
Qué mirar en la Console	El agente pasa a **versión 2**. En el Transcript, la llamada rechazada; en la pestaña **Tools**, las llamadas a `lookup_stock` y a bash.
Tiempo y costo	3 minutos, céntimos.

Módulo 04 · Files

Qué aprendes	Subir archivos (Files API), montarlos en la sesión (`/mnt/session/uploads/`), agregar uno con la sesión en marcha y descargar lo que el agente escribe en `/mnt/session/outputs/`.
Qué ejecutas	Revisión de cinco contratos, luego se agrega el sexto. Se descargan `summary.csv` y `report.md` a `04-files/outputs/`.
Qué mirar en la Console	Pestaña **Resources**: los contratos montados y los dos outputs del agente.
Tiempo y costo	3–4 minutos, céntimos.

Módulo 05 · Session Control

Qué aprendes	Controles para sesiones largas: **presupuesto** (tope duro en centavos), `system.message`, **interrupt** y redirección, **reconectar** sin perder eventos, auditar el historial, y pausar al llegar al tope para luego reanudar.
Qué ejecutas	Una sola sesión pasa por todo. El presupuesto se baja a propósito para ver `budget_reached`; al subirlo, el trabajo sigue solo.
Qué mirar en la Console	Pestaña **Session**: el gasto contra el presupuesto. En **Events**, el `user.interrupt` y el `system.message`.
Tiempo y costo	2–3 minutos, céntimos.

Módulo 06 · Subagents

Qué aprendes	El bloque `multiagent`: un agente líder (Opus) delega en dos especialistas (Haiku), cada uno en su propio **thread**. El líder delega **por nombre**: los nombres del system prompt deben coincidir con los de los agentes.
Qué ejecutas	Revisión de contratos repartida entre el revisor de responsabilidad y el de renovaciones; después, una pregunta de seguimiento al mismo subagente.
Qué mirar en la Console	La **Timeline** con un carril por agente, y la pestaña **Threads** con el costo de cada uno.
Tiempo y costo	4–5 minutos, unos US$0,35 (presupuesto US$3).

Módulo 07 · Dynamic Workflows

Qué aprendes	El objetivo del cookbook: el agente **escribe un programa** (workflow) que lanza muchos agentes por fases, y el servidor lo ejecuta en segundo plano. El trabajo termina cuando no queda ningún run abierto **y** llega un idle con `end_turn`.
Qué ejecutas	Un run con fases (extraer, verificar todas las extracciones, rehacer las que fallen): un lector Haiku por contrato y verificadores inline. Al final se le pide al agente que imprima el workflow que escribió.
Qué mirar en la Console	La Timeline se llena de carriles: más de una docena de threads. En **Threads**, el costo de cada lector y verificador.
Tiempo y costo	3–5 minutos, alrededor de US$1 (presupuesto US$5).

Las versiones -cli hacen lo mismo con `ant`. La primera vez que ejecutas `ant apply`, imprime el ID de tu organización y de tu workspace: no lo compartas en capturas. Si lo vuelves a ejecutar, responde *"Everything is up to date"*: no duplica recursos.

8. Reglas de oro

Regla	Por qué
Stream-first	Un stream solo entrega los eventos emitidos después de abrirse.
Un idle no es el final	Mira `stop_reason`: `end_turn`, `requires_action`, `budget_reached`… Con workflows, espera además a que se cierren todos los runs.
Responde cada llamada al llegar	Con varios agentes, la sesión puede seguir en `running` mientras uno espera tu respuesta.
Crea una vez, reutiliza IDs	Environments y agentes son configuración; las sesiones son ejecuciones.
Pon presupuesto	Sobre todo con subagentes y workflows: un solo tope cubre a todos los agentes.
Descarga antes de borrar	Borrar una sesión elimina también los archivos que produjo.

9. ¿Y después?

- **Escala los datos:** pon cientos de contratos en `sample-data/` y mira cómo el run del módulo 07 reparte el trabajo.
- **Define outcomes:** un evaluador revisa el entregable contra una rúbrica y pide revisiones hasta que cumple.
- **Webhooks y scheduled deployments:** recibe un aviso cuando una sesión larga termina, o ejecuta una revisión como esta de forma periódica.

Documentación oficial: `platform.claude.com/docs/en/managed-agents/overview` · Repo: `github.com/davila7/claude-managed-agents-cookbook`